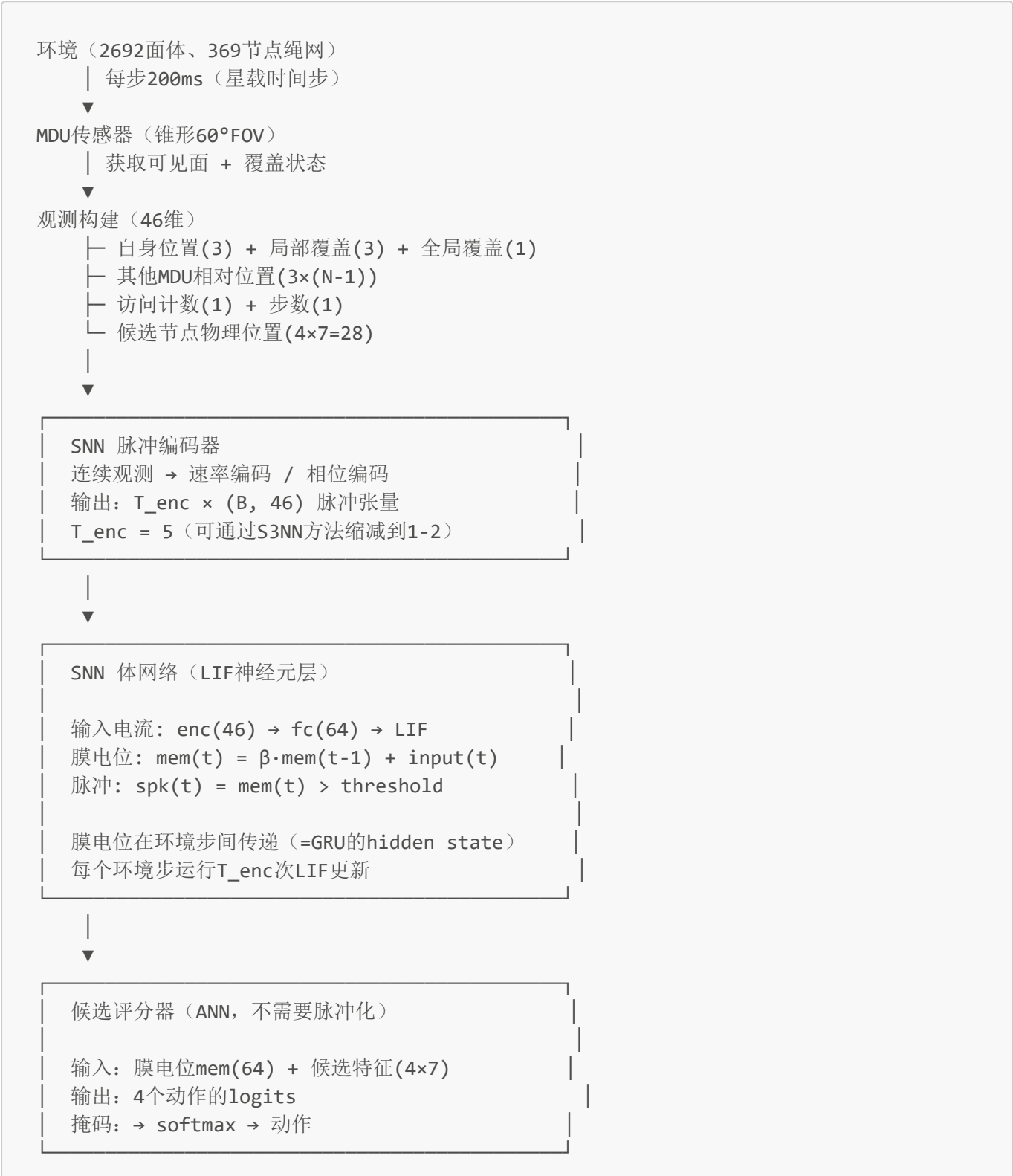


SNN 时序轨迹规划 — 详细实施方案（中文版）

假设前提：SNN可以跑通且效果良好。**目标平台：** 航天级神经形态芯片（FPGA、Loihi类）。**地面仿真：** PyTorch + snnTorch + surrogate gradient。

一、总体架构

1.1 信息流



↓
MDU移动到选中邻居节点

1.2 LIF 神经元数学模型

核心公式（每个时间步）：

$I(t) = W \cdot x(t) + b$	← 输入电流
$mem(t) = \beta \cdot mem(t-1) + I(t)$	← 膜电位积分
$spk(t) = \Theta(mem(t) - V_{th})$	← 脉冲发放 (Θ =阶跃函数)
$mem(t) = mem(t) \cdot (1 - spk(t))$	← 脉冲后重置

其中：

- β = 膜电位泄漏率 (0~1)，可学习
- V_{th} = 发放阈值 (通常=1)
- $spk(t) \in \{0, 1\}$ ，脉冲事件
- Surrogate gradient：用 fast_sigmoid 近似 Θ 的梯度

1.3 与当前GRU的直接对比

ANN+GRU: $h(t) = GRU(x(t), h(t-1))$ hidden state = $h(t)$ GRU方程: $z = \text{sigmoid}(W_z \cdot [x, h])$ $r = \text{sigmoid}(W_r \cdot [x, h])$ $n = \tanh(W_n \cdot [x, r \cdot h])$ $h' = (1-z) \cdot n + z \cdot h$ ~12个权重矩阵	SNN+LIF: $mem(t) = \beta \cdot mem(t-1) + W \cdot x(t)$ $spk(t) = \Theta(mem(t) - V_{th})$ hidden state = $mem(t)$ (膜电位) LIF方程: $I(t) = W \cdot x(t)$ $mem = \beta \cdot mem + I$ $spk = mem > V_{th}$ ~1个权重矩阵	← 更简单! ← 3个参数 vs 12个 ← 事件驱动
---	--	--

LIF 比 GRU 简单一个数量级，但通过脉冲时序编码信息。

二、脉冲编码方案

2.1 速率编码（Rate Coding）— 推荐入门

```
def rate_encode(obs, T_enc=5):
    """连续值 → 泊松脉冲序列"""
    probs = torch.clamp(obs, 0, 1) # 归一化到[0,1]作为发放概率
    spikes = []
    for t in range(T_enc):
```

```

    spike = torch.bernoulli(probs) # 每个维度独立泊松采样
    spikes.append(spike)
    return torch.stack(spikes) # (T_enc, B, obs_dim)

```

优点： 简单、概率解释清晰 **缺点：** T_{enc} 需要足够大以编码精度；采样噪声

2.2 相位编码 (Phase Coding) — 推荐高效

```

def phase_encode(obs, T_enc=5, T_ref=1.0):
    """连续值 → 基于相位的单脉冲"""
    # 每个维度在特定相位发放一次脉冲
    # 值越大 → 越早发放
    phase = 1.0 - torch.clamp(obs, 0, 1) # [0,1] → [1,0]
    spike_time = (phase * (T_enc - 1)).long() # 量化为离散时间步
    spikes = torch.zeros(T_enc, *obs.shape)
    for t in range(T_enc):
        spikes[t, spike_time == t] = 1.0
    return spikes

```

优点： 每个维度每步最多1个脉冲，能量效率高 **缺点：** 精度受 T_{enc} 限制

2.3 群体编码 (Population Coding) — 推荐精度

来自 Tang et al. 2020 的方法：

```

def population_encode(obs, n_neurons=16):
    """每个连续值用一组神经元编码"""
    # 每个神经元有优选值 $\mu_i$ 
    # 发放率 =  $\exp(-(\text{obs} - \mu_i)^2 / \sigma^2)$ 
    centers = torch.linspace(-1, 1, n_neurons)
    rates = torch.exp(-((obs.unsqueeze(-1) - centers) ** 2) / 0.1)
    return rates

```

优点： 编码精度高、生物可解释 **缺点：** 维度膨胀 (46维 × 16神经元 = 736维)

三、训练流程

3.1 PPO with SNN 伪代码

```

for episode = 1 to N:
    重置环境
    重置SNN膜电位 → mem = zeros

    for step = 1 to max_steps:
        # 脉冲编码
        obs_spikes = rate_encode(obs, T_enc=5) # (5, B, obs_dim)

```

```
# SNN前向（T_enc个内部时间步）
for t_enc in range(T_enc):
    cur = enc_fc(obs_spikes[t_enc])      # 输入电流
    spk, mem = lif(cur, mem)             # LIF更新

# 候选评分（ANN，用膜电位）
body_feats = mem
scores = scorer(body_feats, candidates)
action = Categorical(softmax(scores)).sample()

# 环境步
obs', reward, done = env.step(action)
store(obs, action, reward, ...)

# PPO更新
compute_gae()
for epoch in range(K):
    # 重新SNN前向（梯度通过surrogate gradient传播）
    # 截断BPTT，每16个环境步截断
    ...
    loss = PPO_clip_loss + entropy_bonus
    loss.backward() # 梯度自动通过LIF的surrogate gradient传播
    optimizer.step()
```

3.2 Surrogate Gradient 选择

snnTorch 提供多种 surrogate gradient 函数：

函数	表达式	特点
fast_sigmoid(slope=25)	$\sigma'(x) \approx \sigma(x)(1-\sigma(x)) \cdot \text{slope}$	最常见，稳定
atan	$\text{atan}'(x) = 1/(1+x^2) \cdot \text{slope}$	梯度平滑
straight_through	前向：阶跃；反向：直通	最简单的近似
triangular	三角形窗	局部梯度高

推荐：fast_sigmoid(slope=25) — Tang et al. 2020 验证过与PPO兼容。

3.3 截断BPTT实现

SNN的BPTT与GRU的BPTT非常相似：

```
# 每个epoch
mem = zeros(A, H) # 重置膜电位
for t in range(T): # T = max_steps
    for chunk in range(0, T, BPTT_LEN):
        chunk_end = min(chunk + BPTT_LEN, T)

        for t_env in range(chunk, chunk_end):
```

```
# 内部SNN时间步
for t_enc in range(T_enc):
    spk, mem = lif(cur, mem)

# PPO损失
loss = ppo_loss(probs, actions, advantages)

if t_enc == chunk_end - 1 and t_enc == T_enc - 1:
    loss.backward() # 块边界: 不保留图
else:
    loss.backward(retain_graph=True)

# 块边界: clip + step + detach
clip_grad_norm()
optimizer.step()
mem = mem.detach() # 截断BPTT
```

3.4 ANN→SNN权重迁移

这是训练SNN的"捷径":



为什么有效： ANN的权重编码了有效的覆盖策略。SNN的LIF层在膜电位中模拟了类似GRU的动力学。权重迁移后只需微调SNN特有的时间参数（β、阈值）。

四、部署到航天平台

4.1 目标平台对比

平台	功耗	成熟度	航天级	适合
Intel Loihi 2	~1W	量产	✗ 未认证	原型验证
Xilinx Kintex FPGA (Rad-Tol)	~3W	成熟	✓ 有航天级	推荐主要目标
Microchip RTG4 FPGA	~2W	成熟	✓ 航天级	备选
定制模拟芯片	~10mW	研究	潜力	长期目标

推荐：Xilinx Kintex 抗辐射FPGA — 现有航天级产品，可实现LIF推理。

4.2 FPGA部署流程

训练后量化：

- float32 → int8 权重
- LIF阈值、β量化

硬件映射：

- LIF神经元 → FPGA LUT + DSP
- 脉冲事件 → 稀疏事件驱动

接口：

- 事件相机 → SNN输入
- SNN输出 → MDU电机驱动

4.3 星载优势具体分析

航天需求	ANN+GRU方案	SNN方案	优势
功耗预算（典型~10W）	GPU: 30-75W ❌	FPGA: 1-3W ✅	10-30倍
单粒子翻转（SEU）	全计算错误 ❌	脉冲事件可自恢复 ✅	天然鲁棒
实时性（200ms步）	GPU延迟不稳定 ❌	FPGA确定延迟 ✅	可预测
长期自主	需地面更新 ❌	片上STDP/E-prop ✅	在轨适应
热管理	GPU需要散热 ❌	FPGA被动散热 ✅	系统简化

五、实验验证路线

Phase 0: 环境准备（~30分钟）

```
pip install snntorch
python -c "import snntorch as snn; print(f'snnTorch OK: {snn.__version__}')"

```

Phase 1: 5 episodes 快速测试（~2分钟）

- 验证SNN Actor前向/反向传播
- 检查脉冲发放率（正常范围：0.1-0.5）
- 检查膜电位范围（不应饱和到±∞）

Phase 2: 200 episodes 完整训练（~2小时）

- 对比GRU vs SNN覆盖率
- 对比GRU vs SNN熵收敛
- 记录脉冲发放率随训练的变化

Phase 3: T_enc 消融实验 (~2小时)

- T_enc = 1, 2, 3, 5, 10 → 覆盖率对比
- 找最小有效T_enc (目标: ≤3)

Phase 4: ANN→SNN迁移 (~1小时)

- 加载当前GRU权重 (88.04%)
- 微调SNN
- 对比直接训练 vs 迁移学习

Phase 5: 多MDU扩展 (~2小时)

- SNN + CTDE参数共享
- 多智能体脉冲通信
- 对比注意力机制

六、已知风险与缓解

风险	概率	影响	缓解
SNN训练不收敛	中	高	ANN→SNN迁移预训练
Surrogate gradient不稳定	中	中	降低LR, 增加熵系数
T_enc太长导致仿真慢	低	中	S3NN方法缩减时间步
FPGA实现精度损失	低	低	量化感知训练
脉冲发放率过低/过高	中	中	正则化项约束发放率

七、参考文献

详见 [papers-full-list.md](#) 和 [README-zh.md](#)。